COE 528 - FINAL PROJECT REPORT

Instructor: Boujemaa Guermazi
TA: Luella Marcos
Name: Akarshan R Singh Akarshan R Singh
Student ID: 501273538
Section: 01(Tuesday)

Introduction

This report presents the design and implementation of a **JavaFX-based Bank Account Application**, incorporating essential functionalities for two distinct user roles: **Managers** and **Customers**. The application supports customer operations such as logging in, managing account balances, and executing online transactions with varying fees based on account levels **(silver, gold, platinum)**. Managers have administrative capabilities including user management and account creation. The system utilizes **UML diagrams** to model its architecture and implements the **State design pattern** to dynamically manage customer levels. Detailed documentation includes **use case and class diagrams**, along with a technical description of key classes and methods, ensuring robust application functionality and adherence to design principles [1].

Use Case Diagram Description

The **Use Case Diagram** for the **Banking Management System** illustrates interactions between the **Manager** and **Customer** actors with various system functionalities. Key use cases include **Add Customer**, **Delete Customer**, **Deposit Money**, **Withdraw Money**, **Make Online Purchase**, and **Log Out**. **Authenticate User** is central, linking to multiple use cases through **include** relationships, signifying essential actions like authentication. **Extend** relationships show optional actions, such as **Handle Invalid Credentials** extending **Log In**, and **Notify Insufficient Funds** extending **Withdraw Money**. This diagram effectively outlines the system's functional requirements and actor interactions [4].

Use Case Textual Description

Attribute [6]	**Details** [6]
Use Case name	Withdraw Money
Participating Actors	Performed by Customer
Entry Conditions	• Customer is authenticated and logged into the banking system.

	● Customer has an active account.
Flow of Events	1. Customer selects "Withdraw Money" from the account menu. 2. System prompts Customer to enter a withdrawal amount. 3. Customer inputs and confirms withdrawal amount. 4. System verifies account balance against the requested amount. 5. If sufficient funds: ● System deducts the specified amount from the Customer's account. ● System updates the account balance. ● System records the transaction in the transaction log. ● System provides a confirmation of the successful withdrawal to Customer. 6. If insufficient funds: ● System cancels the transaction. ● System triggers the "Notify Insufficient Funds" use case.
Exit Conditions	● Customer's account balance is updated to reflect the withdrawal. ● Transaction confirmation is displayed to the Customer.
Exceptions	● Invalid amount entered (negative or non-numeric): System displays an error message and prompts for a valid amount. ● Insufficient funds: System cancels the transaction and triggers the "Notify Insufficient Funds" use case.
Special Requirements	● Real-time balance verification. ● Secure and accurate transaction logging for auditing purposes. ● Intuitive user interface for seamless user interaction.

Class Diagram Description

The **UML class diagram** models a **banking system** with core entities and their interactions. The **Bank class** acts as the main interface, managing screens for **customers and managers**. The **Customer** class, representing **clients**, includes attributes for **balance**, **login status**, and **aggregates User** and **Level** objects. The **User** class stores user-specific information such as **username**, **password**, **role**, and **balance**. The **Manager** class **extends User**, adding capabilities to manage customers. The **Level class** is an **abstract** base class with attributes and methods for handling customer levels, **inherited** by **Silver**, **Gold**, and **Platinum** subclasses, each defining specific level behaviors and fees. The design employs **inheritance** to define common interfaces and behaviors among Level subclasses, **aggregation** in the **Customer** class to establish a **"whole-part"** relationship, and **composition** in the **Bank** class to encapsulate and manage

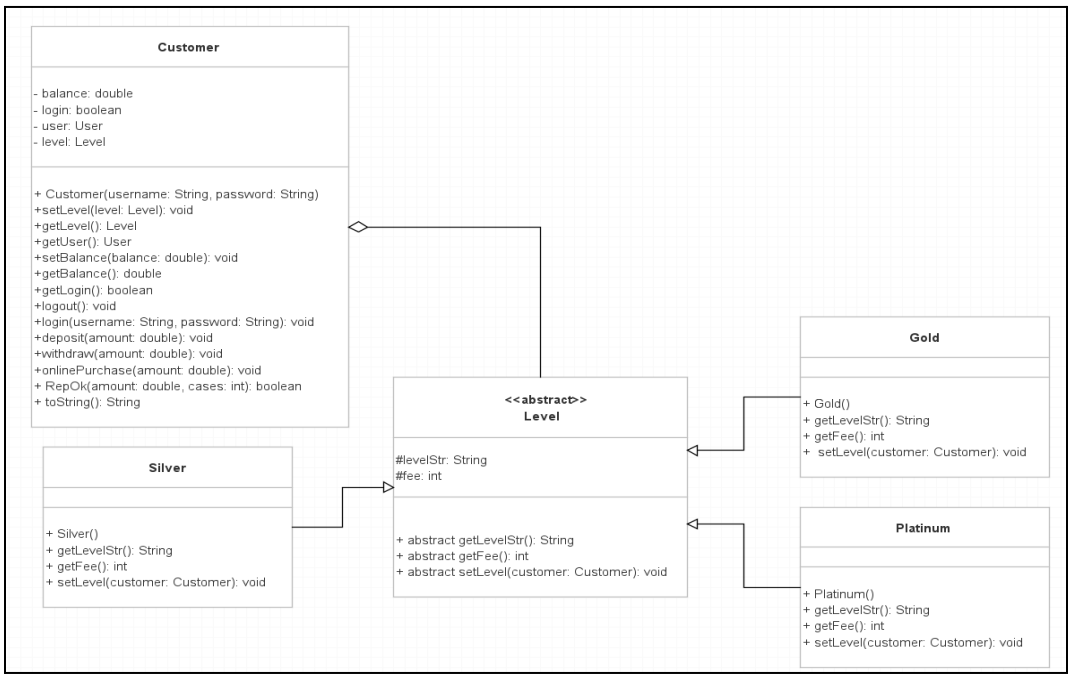
interactions and dependencies of **Customer**, **Manager**, and **User**, ensuring a flexible and robust system [4].

Implementation and Verification of Customer Class to Meet Project Requirements

The functionality of the **Customer class** aligns with the **requirements** outlined in **Point 2** of the project, focusing on managing **customer accounts**, including deposit, withdrawal, and purchase operations, and tracking customer levels based on their balance. This class is **mutable**, allowing **modifications** to the **balance**, **level**, and **login status** [2]. The class includes an **overview** clause [2] and defines an **abstraction function (AF(c))** and **rep invariant (RV(c))** [3]. Each method is documented with **requires**, **modifies**, and **effects** clauses, ensuring clarity on preconditions, state modifications, and outcomes [2]. The **toString()** method implements the **abstraction function**, providing a **string representation** of the Customer object, while the **RepOk()** method verifies the **rep invariant**, ensuring the **integrity** of the object's state [3]. This comprehensive approach ensures the **Customer class** meets the project requirements, effectively managing **customer account** operations and maintaining system consistency [1].

State Design Pattern in the UML Class Diagram

The **State design pattern** is represented in the UML class diagram through the **Customer** and **Level classes** [1]. The **context class** is **Customer**, which holds a reference to an instance of the **state class Level**, allowing the customer's state to be represented dynamically. The **Level class** is **abstract**, defining the interface for various states, and it is implemented by the **concrete classes Silver, Gold, and Platinum**. Each subclass provides specific implementations for methods such as **getLevelStr(), getFee(), and setLevel(Customer)** [4]. This structure allows the Customer class to change its behavior based on its current level by assigning different Level instances, thus encapsulating state-specific behavior [4]. For instance, upgrading a customer's level involves changing the Level reference from **Silver to Gold**, which modifies the applicable fees and privileges without altering the Customer class logic [7]. This pattern allows an object to **alter its behavior** when its internal state changes; the object will appear to change class. This pattern enhances the system's **flexibility** and **scalability** by allowing easy addition of new levels and their behaviors [4].



References:

[1] Project Manual for COE 528. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[2] Lecture 4 _ Software Engineering - Abstraction and Decomposition. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[3] Lecture 5_ Procedural Abstraction - Data Abstraction. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[4] Lecture 6_ Modeling with UML. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[5] Lecture 7_ Design Patterns - Part1. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[6] COE 528: Final Project TA Notes. Department of Computer Engineering, Toronto Metropolitan University, 2024.

[7] State Design Pattern Diagram. Violet UML Editor, 2024.